Database

“Life is a long lesson in humility.”
- James M. Barrie



Outline

1. Limitation
2. Partitioning/Sharding
3. How to choose the right DB?

Limitation on a Single DB

Limitation on a Single DB

- With mid-level hardware and a new version DB:
- **~ 20K QPS**
- TPS (based on TPC-C):
  - Low contention: **~1000** (Postgres Reference)
  - High contention: ~200 (based on my exp)
- With low-level hardware and old version:
  - MySQL 5.7 & a cloud server (4 cores, 8G)
  - ~ 10K QPS
  - ~ 500 TPS
- Note: **Based on estimation and limitations to design**

How to Handle With a Big Table?

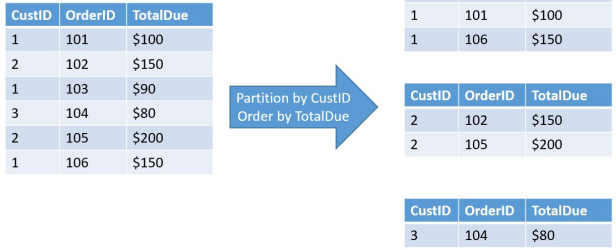
How to Handle With a Big Table?

- Evaluate the data size, access patterns first
- Do not jump into complex solution first
- Start from simple solutions:
  - Rewrite query
  - Index
  - Archive old data
  - Caching
  - Using multi-process features maximally
  - Master - Slave
  - Partitioning / Sharding
  - Other Database

Partitioning
Sharding

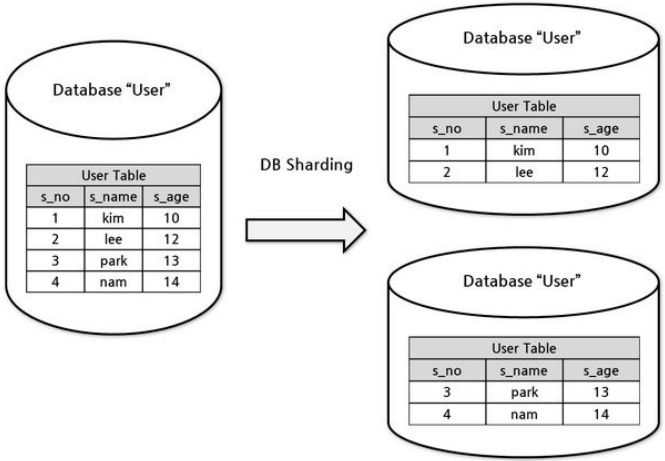
Pros

- **Improve Query Performance**
- **Fast to remove partitions**
- Easy to migrate partitions to cheaper storage (native remote partitions)



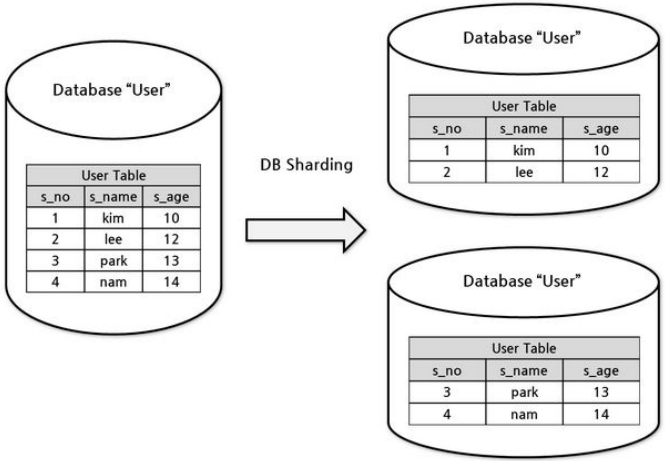
Cons

- Limation: **Relational DB: all unique keys (or PK) on a partitioned table must include the partitioning key.**
  - the partition keys must not include any expressions or function calls
- Secondary index
- **Scatter Query**
  - Note: queries must include the partition key to avoid Scatter Query
- **(Shard) Transactions don't work across shards.**
- Hard to join
- Routing
- Foreign Ref to the partitioned table
- Rebalancing



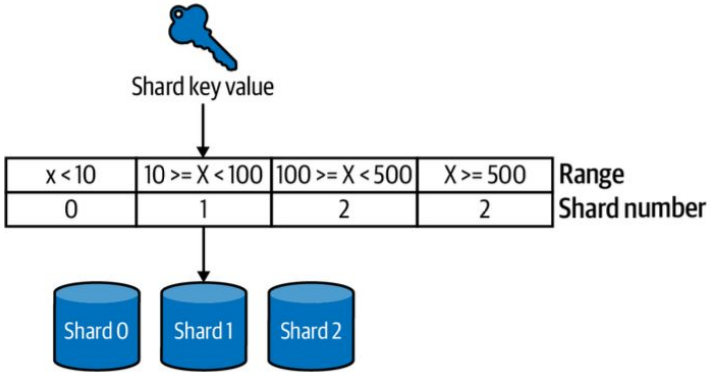
When to partition / shard

- It’s hard to define when. The following is suggestion.
- Large Table
  - >= 50M records: partition
  - >= 200M records: shard
- Write rate is very high: >= 3000 TPS
- Query Performance Degrade: Popular query P(99) > 120ms
- Maintenance Overhead: VACUUM, Indexing take longer



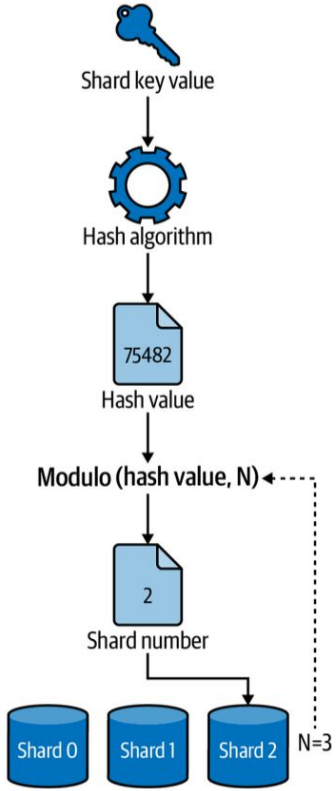
Range Partitioning

- There is no overlap between the ranges.
- Pros
  - Simple
  - Align with biz logic
- Cons
  - Fine tune to get even distribution
  - **Hot spot**
- Example: a news feed system
  - The `post` table is partitioned by `created_date`
  - The 98% traffic goes to the last partition



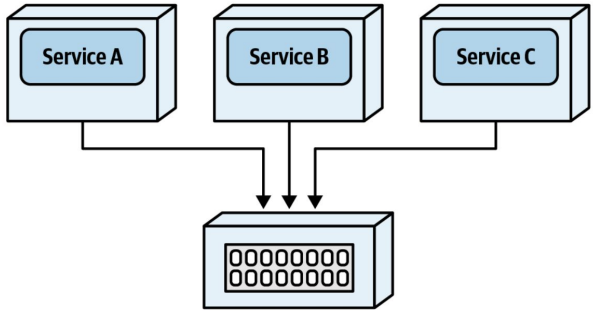
Hash Partitioning

- Pros
  - **Even Distribution on both storage & operations**
- Cons
  - **Inefficient for range queries**
  - Unusual hot spot
- My opinion: Hash partitioning is preferred.



Partition Key

- **Common Data Access Pattern + Strategy → Partition Key**
- An ideal key has some of 3 properties:
  - Reference entity → to avoid accessing across shards
  - Small → because it's heavily used
  - High cardinality → even distribution



Partition Key / Example 1

- Context:
  - System: E-Commerce
  - Table: Order
- What are common queries?
  - Insert orders
  - Order history of a user
  - Order detail
- What are the appropriate partition strategy and partition key? What is the primary key?
  - **HASH (user_id)**
  - The primary key: order_id, user_id

Partition Key / Example 2

- Context:
  - System: Payment System
  - Table: Transaction
- What are common queries?
  - Insert transactions
  - Transaction history for both debit account & credit account
  - View Transaction detail
- What are the appropriate partition strategy and partition key? What is the primary key?
  - **Should not be partitioned**
  - The primary key: transaction_id

Partition Key / Example 3

- Context:
  - System: Metric monitoring system
  - Table: Metric_log
- What are common queries?
  - Insert metrics from a device at specific time
  - Show history metric log of a device
  - Delete old metrics
- What are the appropriate partition strategy and partition key? What is the primary key?
  - **Nested partitions**
    - Level 1: HASH(device_id)
    - Level 2: Range(timestamp)
  - Primary key: (device_id, timestamp)
- It’s hard to maintain and inefficient → Time-series DB

Partition Key / Example 4

- Context:
  - System: Discord system
  - Table: message
- What are common queries?
  - Insert messages
  - View message history in a channel
- What are the appropriate partition strategy and partition key? What is the primary key?
  - HASH (channel_id)
  - The primary key: (message_id, channel_id)
  - **Sort by message_id (not created_at)**

Best Practices

- If there is no performance bottleneck, **try not to shard the database**.
- If you want to do it, try to do it in **one go**. For example, 16 database and 64 partitions can basically meet your business needs for the next few years.
- Many NoSQL databases provide auto sharding features. You can also **consider using these NoSQL** databases to replace traditional relational databases.

Alternatives

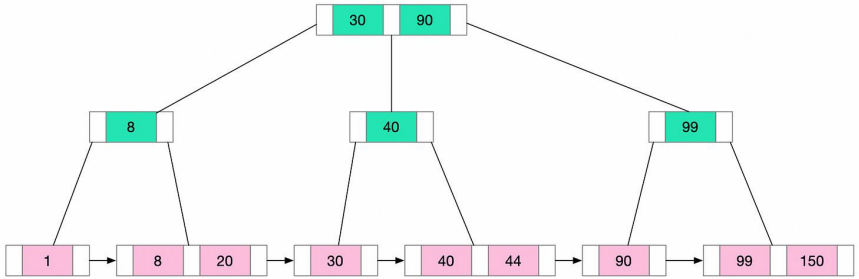
- **NoSQL**
- NewSQL
  - Example: **TiDB**, Cockroachdb
  - Cons:
    - Not full ANSI SQL
    - OLAP need to be improved more
- Middleware
  - Example: Vitess (MySQL), Citus (Postgresql)
  - Cons:
    - Operation overhead

Rating subject	RDBMS databases (Oracle, SQL Server, Postgres, etc.)
Ease of learning	★★★★
Ease of data modeling	★★★
Scalability/throughput	★★
Availability/partition tolerance	★
Consistency	★★★★★
Programming language support, product maturity, SQL support, community	★★★★
Read/write priority	

Why Relational Database is Bad At Scalability?

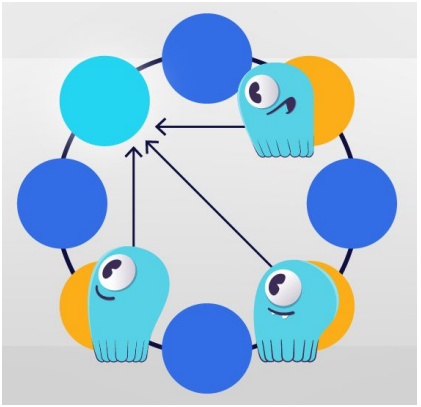
Why Relational DB is not good at Scalability?

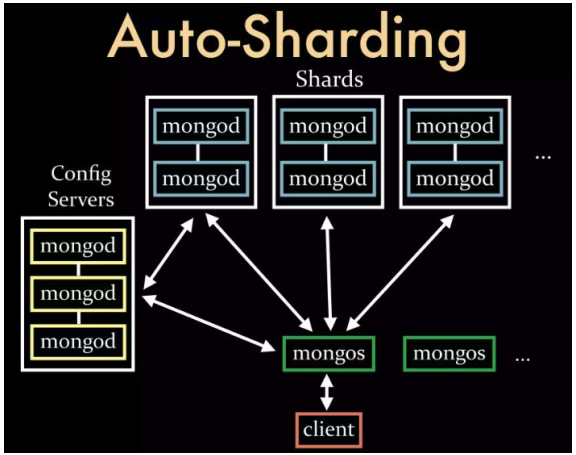
- Strong Consistency
- Monolithic design: easy to join, but hard to scale
- **Random I/O**:
  - Fact: Sequential I/O >> Random I/O
  - To insert a record, we scan B+Tree index to find the location
    → generate many random I/O
- Lack of auto-shard feature



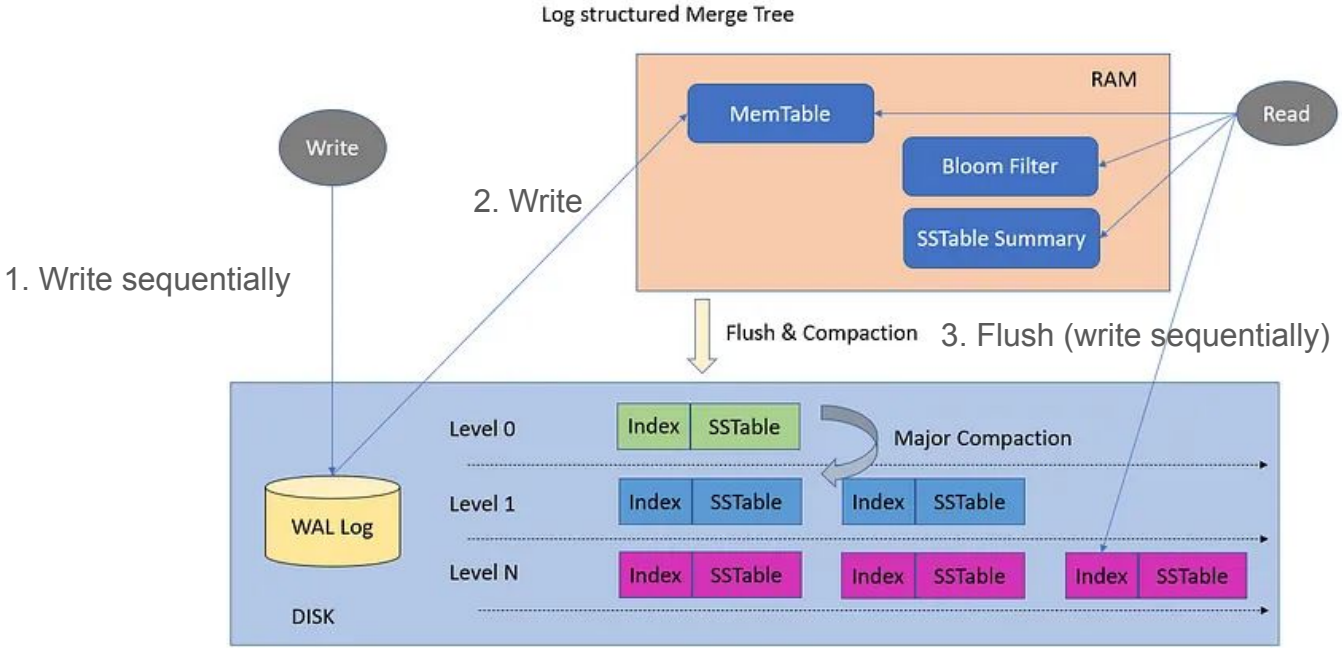
Why NoSQL DB is good at Scalability?

- Distributed design at the beginning
- **LSM Tree: convert random I/O to sequential I/O**
- Built-in Auto-shard feature (MongoDB)



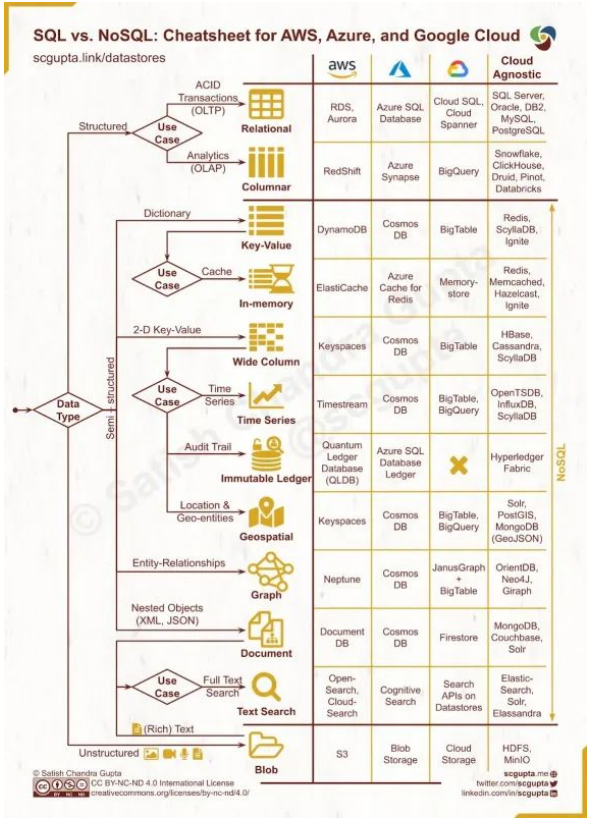


LSM Tree

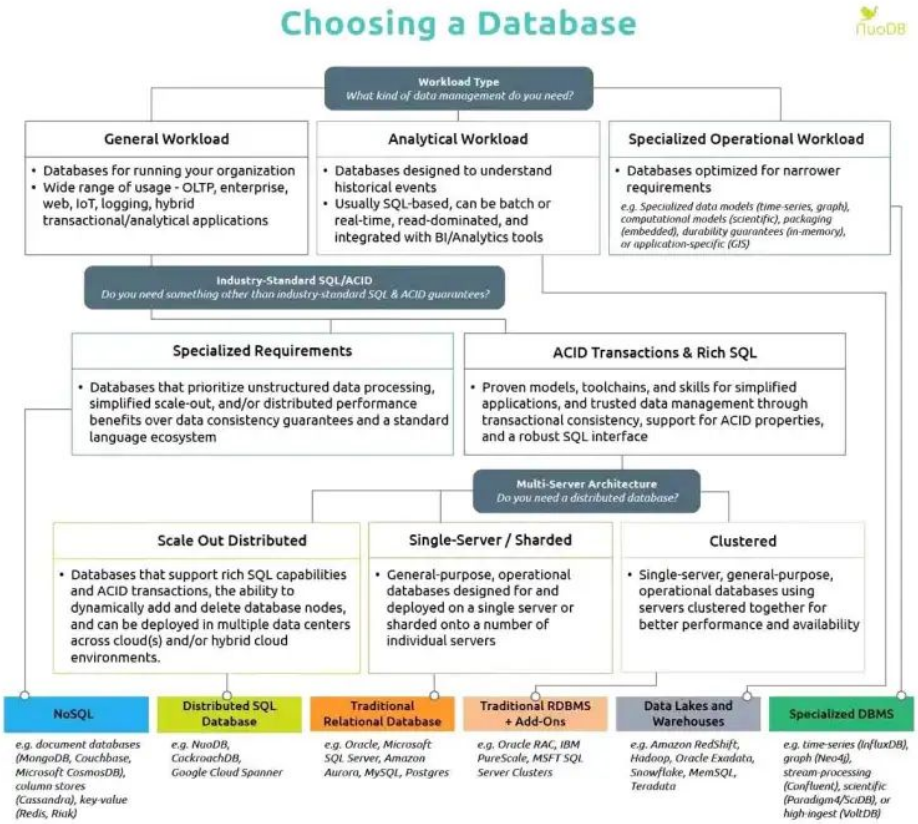

Log structured Merge Tree
RAM
MemTable
Read
Write
Bloom Filter
2. Write
SSTable Summary
1. Write sequentially
Flush & Compaction
3. Flush (write sequentially)
Level 0
Index
SSTable
Major Compaction
Level 1
Index
SSTable
Index
SSTable
WAL Log
Level N
Index
SSTable
Index
SSTable
Index
SSTable
DISK

How to choose the right DB?

How to choose the right DB?

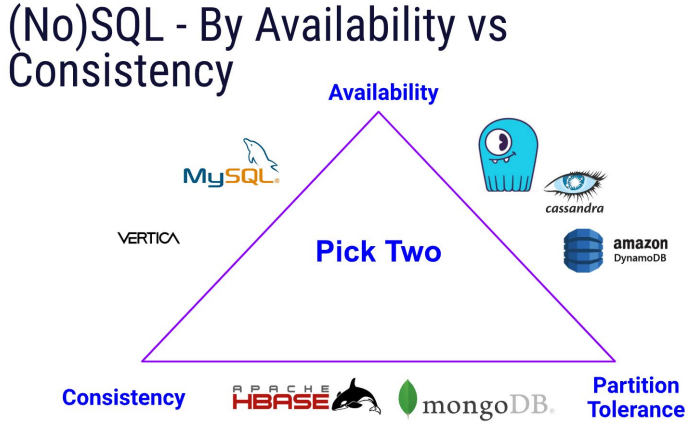


How to choose the right DB?



How to choose the right DB?

- High performance for write + read operation
- Scalability
- No need for strong consistency
- No special function on data
- If you need a flexible data model & complex queries
  - → MongoDB
  - UC: Device info, IoT metrics
- **If you need performance and scalability**
  - → ScyllaDB
  - UC: messages, posts



Complexity

Key / Value	RocksDB, AEROSPIKE, redis
Document store	mongoDB., Couchbase
Wide column store	cassandra, amazon DynamoDB, APACHE HBASE
Graph	JanusGraph, neo4j

Column-Oriented Storage

- Usually used for OLAP (online analytic processing)
- Each column file contains the row value in the same order

fact_sales table

date_key	product_sk	store_sk	promotion_sk	customer_sk	quantity	net_price	discount_price
140102	69	4	NULL	NULL	1	13.99	13.99
140102	69	5	19	NULL	3	14.99	9.99
140102	69	5	NULL	191	1	14.99	14.99
140102	74	3	23	202	5	0.99	0.89
140103	31	2	NULL	NULL	1	2.49	2.49
140103	31	3	NULL	NULL	3	14.99	9.99
140103	31	3	21	123	1	49.99	39.99
140103	31	8	NULL	233	1	0.99	0.99

Columnar storage layout:

date_key file contents:	140102, 140102, 140102, 140102, 140103, 140103, 140103, 140103
product_sk file contents:	69, 69, 69, 74, 31, 31, 31, 31
store_sk file contents:	4, 5, 5, 3, 2, 3, 3, 8
promotion_sk file contents:	NULL, 19, NULL, 23, NULL, NULL, 21, NULL
customer_sk file contents:	NULL, NULL, 191, 202, NULL, NULL, 123, 233
quantity file contents:	1, 3, 1, 5, 1, 3, 1, 1
net_price file contents:	13.99, 14.99, 14.99, 0.99, 2.49, 14.99, 49.99, 0.99
discount_price file contents:	13.99, 9.99, 14.99, 0.89, 2.49, 9.99, 39.99, 0.99

Column-Oriented Storage

- `product_sk` column: low cardinality
- Query:

SELECT count(*)

FROM sales

WHERE product_sk IN (29, 30, 74)

OR 3 bitmap → count the number of “1” in the result bitmap is so quick (hardware support)

Column values:

product_sk:	69	69	69	69	74	31	31	31	31	29	30	30	31	31	31	68	69	69

Bitmap for each possible value:

product_sk = 29:	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
product_sk = 30:	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
product_sk = 31:	0	0	0	0	0	1	1	1	1	0	0	0	1	1	1	0	0	0
product_sk = 68:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
product_sk = 69:	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
product_sk = 74:	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Column-Oriented Storage

- Compression: run-length encoding
  → pro: save storage
- Problem: It's hard to update-in-place with compression columns.
- Solution:
  - All writes go to an in-mem store first
  - Batch insert (Clickhouse)

Column values:

product_sk:	69	69	69	69	74	31	31	31	31	29	30	30	31	31	31	68	69	69

Bitmap for each possible value:

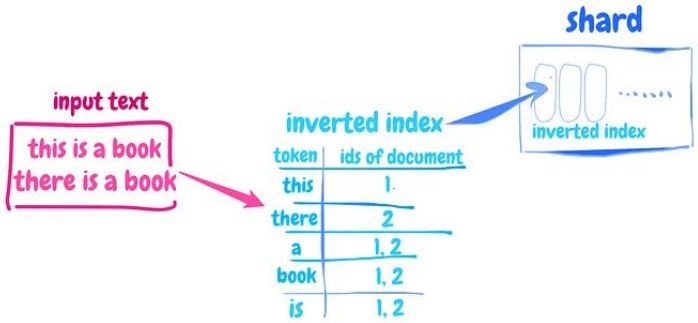
product_sk = 29:	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
product_sk = 30:	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0
product_sk = 31:	0	0	0	0	0	1	1	1	1	0	0	0	1	1	1	0	0	0
product_sk = 68:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
product_sk = 69:	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
product_sk = 74:	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Run-length encoding:

product_sk = 29:	9, 1	(9 zeros, 1 one, rest zeros)
product_sk = 30:	10, 2	(10 zeros, 2 ones, rest zeros)
product_sk = 31:	5, 4, 3, 3	(5 zeros, 4 ones, 3 zeros, 3 ones, rest zeros)
product_sk = 68:	15, 1	(15 zeros, 1 one, rest zeros)
product_sk = 69:	0, 4, 12, 2	(0 zeros, 4 ones, 12 zeros, 2 ones)
product_sk = 74:	4, 1	(4 zeros, 1 one, rest zeros)

Search Engine

- Elasticsearch (ES)
- Why ES Search So Fast?
  - **Inverted Index**
  - Parallel processing on shards/segments
  - Filesystem cache
  - Smart Relevance Scoring to avoid scanning unnecessary data

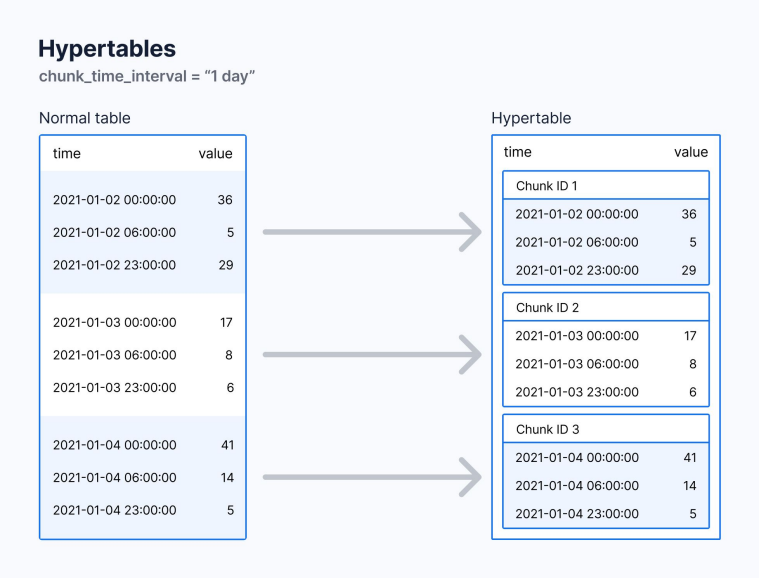


Time series Databases

Rating subject	Time series databases (InfluxDB, TimescaleDB, etc.)
Ease of learning	★
Ease of data modeling	★★
Scalability/throughput	★★★★
Availability/partition tolerance	★★
Consistency	★★★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	

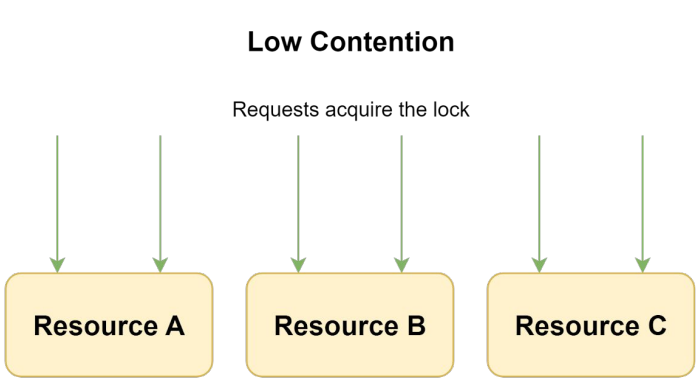
Time Series Databases

- Timescale (recommended)
- A hypertable automatically partitions data by time/space
- Each hypertable is made up of child tables called chunks.
- Improve performance for write heavy
  - Write to mem first
  - Batch Commit
  - Parallelize writes across many servers
  - synchronous_commit=off
- Read more:
  - TimescaleDB vs. InfluxDB: Purpose Built Differently for Time-Series Data

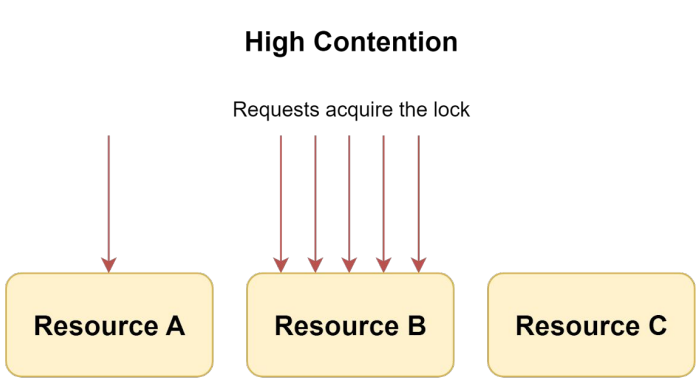


Contention

Contention



Strong Consistency
+ High Concurrency
+ Low Contention:
→ NewSQL
(TiDB: recommended)



Strong Consistency
+ Normal Concurrency
+ High Contention:
→ Traditional Relational DB + Optimistic Lock + other optimization (has limitations)
+ High Concurrency
→ Other solution (discuss later)

Recap

- Evaluate the data size, access patterns first
- Sharding has many cons
- Understanding Principle of Internal Operation help us:
  - Choose the right
  - Control it easily
  - Get insights to solve other problems

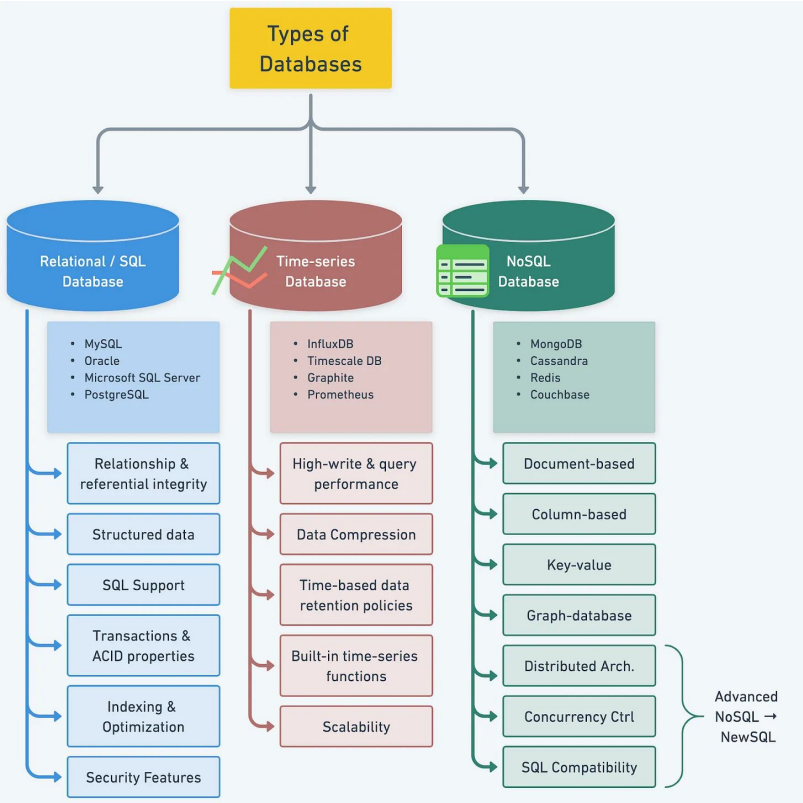
Homework

- Research Rate Limiting Algorithms
  - Which one is preferred?

References

- https://blog.lawrencejones.dev/state-machines/index.html
- https://vladmihalcea.com/optimistic-vs-pessimistic-locking/
- https://faculty.kutztown.edu/schwesin/spring2022/csc343/lectures/Locking.pdf

How to choose the right DB?



Key Value Databases

Rating subject	Key value databases (Redis, DynamoDB, Riak, etc.)
Ease of learning	★★★
Ease of data modeling	★
Scalability/throughput	★★★★
Availability/partition tolerance	★★★★
Consistency	★★
Programming language support, product maturity, SQL support, community	★★★
Read/write priority	

Document Databases

Rating subject	Document databases (MongoDB, CouchDB, Marklogic, etc.)
Ease of learning	★★★
Ease of data modeling	★★★
Scalability/throughput	★★
Availability/partition tolerance	★★★
Consistency	★★
Programming language support, product maturity, SQL support, community	★★★
Read/write priority	

Wide Column Databases

Rating subject	Column family databases (Cassandra, Scylla, Druid, etc.)
Ease of learning	★★
Ease of data modeling	★
Scalability/throughput	★★★★
Availability/partition tolerance	★★★★
Consistency	★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	

Graph Databases

Rating subject	Graph databases (Neo4J, Infinite Graph, Tigergraph, etc.)
Ease of learning	★
Ease of data modeling	★★
Scalability/throughput	★★★
Availability/partition tolerance	★★★
Consistency	★★★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	

NewSQL Databases

Rating subject	New SQL databases (VoltDB, NuoDB, ClustrixDB, etc.)
Ease of learning	★★★
Ease of data modeling	★★★
Scalability/throughput	★★★
Availability/partition tolerance	★★★
Consistency	★★
Programming language support, product maturity, SQL support, community	★★
Read/write priority	

Thank you 🙏

